

Chips That Route Chips

Lineage, Self-Application, and Tools That Help Make Their Successors

Flyxion
Independent Researcher

September 2026

Abstract

A spintronic Ising machine fabricated in a 40-nanometre CMOS process is demonstrated on two problems from electronic design automation, global routing and layer assignment, with a coauthor from an EDA company. The scene invites a reading in which chips have begun to design chips. This essay separates three kinds of self-reference that the reading runs together: token self-application, in which a thing processes its own description; type self-application, in which a thing solves a problem of its own kind; and lineage, in which one generation of tool helps produce the artifacts from which the next generation is made. The Ising chip exhibits the second and, prospectively, the third, but not the first, and at the demonstrated scale the routing problem it solves is one that classical tools already solve exactly and quickly. The more consequential question concerns lineage in general. Machine tools, compilers, and machine-learning models all help make their successors, and their lineages behave differently: flatness can be generated from imperfect plates, a compiler can transmit a property its source code does not show, and a model trained on its predecessors' outputs can lose the tails of its distribution. The difference is traced to whether the lineage contains a check that no generation's output can satisfy by agreeing with itself. Optimization hardware for chip design sits on the favorable side of that line, because its outputs are verified by criteria independent of the solver. Its hazard is not drift but inheritance.

1 A Chip Solving Chip Problems

The paper under discussion demonstrates its Ising machine on “industry-relevant combinatorial optimization problems in electronic design automation: global routing and layer assignment” [1]. The chip that does the solving was itself fabricated in a 40-nm CMOS process, and chips of that kind are laid out, routed, and assigned to metal layers by the same class of tools whose subproblems the Ising machine solves. One of the paper's authors is affiliated with Empyrean Technology, a developer of EDA software. The picture is of a chip solving the problems that had to be solved to make chips, evaluated in part by people who build the tools for solving them.

It is easy to read this as a small instance of self-production, a technology beginning to design itself. The reading is worth taking apart, because what it runs together are three different relations, and only one of them has large consequences.

2 Three Kinds of Self-Reference

Definition 1 (Token self-application). An artifact processes its own description: a compiler compiling its own source, a program printing its own text, a chip routing its own layout.

Definition 2 (Type self-application). An artifact solves a problem of the kind that its own production required, applied to some other instance: a chip routing a net, not its own nets.

Definition 3 (Lineage). Generation n of a tool, T_n , participates in producing an artifact A_{n+1} , from which generation $n + 1$ of the tool is made:

$$T_n \longrightarrow A_{n+1} \longrightarrow T_{n+1}. \quad (1)$$

The Ising chip exhibits type self-application. It routes a net and assigns wire segments to layers, which are the kinds of problem its own fabrication involved, but the instances are demonstration problems, not its own layout. It does not exhibit token self-application, and at 96,000 spins with a demonstrated maximum of 450 coupled spins it could not approach the scale of its own design. It exhibits lineage only prospectively: if Ising accelerators entered EDA flows, they would help route the chips from which later accelerators are made.

Of the three, type self-application is the least remarkable. Every general-purpose computer solves problems of the kind its design required, and every EDA workstation runs on a processor laid out by EDA software. Token self-application is striking but usually a demonstration of completeness rather than a source of new capability. Lineage is the relation that matters, because it is the one through which properties, improvements, and errors pass from one generation to the next.

3 The Scale Gap

Before turning to lineage, the demonstrated problem deserves a closer look, because it bears on how much the scene should be read into.

Global routing is modelled in the paper as a rectilinear Steiner minimum tree problem: connect the pins of a net with rectilinear wires of minimum total length, possibly through auxiliary Steiner points. The general problem is NP-hard [2], which is why an Ising formulation is appropriate in principle. But the difficulty depends on the number of pins, and nets in real designs are overwhelmingly small. Hanan showed that an optimal tree can always be found on the grid formed by the pins' coordinates [3], and FLUTE, a standard tool, computes optimal trees for nets of up to nine pins by table lookup, in time that is effectively constant [4]. The paper's routing instance is defined in the plotting notebook released with it [9]: seven terminals and eight candidate Steiner points, fifteen vertices in all, with an edge variable and an order variable for each ordered pair, giving the $2 \times 15^2 = 450$ spins. A net of seven pins lies well inside the regime where exact classical methods apply, and enumerating every subset of the eight candidate Steiner points with a minimum spanning tree for each finds the optimum in about a hundredth of a second on an ordinary computer. (The companion essay on the paper's released materials reports that this optimum is shorter than the tree the released trajectory reaches [10].)

Industrial global routing is hard not because single small trees are hard but because millions of nets compete for limited routing capacity, and the trees must be chosen jointly under congestion. The Ising demonstration addresses the component that classical tools already solve exactly and quickly at the demonstrated size. This is not a criticism of the demonstration, whose purpose is to show that a constrained EDA problem can be mapped and solved on the hardware. It is a caution against reading the scene as tools already participating in their own production. At present the chip solves a problem of its own kind, at a scale where it is not needed.

4 Lineage

Every real lineage has two sources: the previous generation of tool, and everything else. Writing E_n for the external inputs to generation $n + 1$ (human design decisions, physical materials, measurement standards, new data), the relation is

$$T_{n+1} = \Phi(T_n, E_n). \quad (2)$$

A lineage in which E_n were empty would be self-design in the strong sense. No known technological lineage has empty external inputs, and the interesting questions concern what the lineage does with the part of T_{n+1} that comes from T_n . Three lineages, older than Ising machines, show three different answers.

Plates

A flat surface cannot be made by copying a flat surface, because there is none to copy at the start. Whitworth’s three-plate method generates flatness from plates that are not flat: three plates are scraped against one another in rotation, and the only surfaces that can fit each of the other two in every pairing are planes [5]. Each generation of plate is made with the previous generation, and the lineage converges to a precision that no individual plate possessed at the outset. Much of modern machine-tool accuracy descends from procedures of this kind, in which tools of limited accuracy are used to make tools of greater accuracy.

Compilers

A compiler for a language is typically written in that language and compiled by the previous version of itself. Thompson showed that this lineage can carry a property that no generation’s source code displays [6]. A compiler can be modified to insert a back door when it compiles a login program and to reinsert the modification when it compiles a compiler; once the modified binary exists, the modification can be removed from the source and will persist in every descendant. The lineage transmits a property invisibly, through the binaries that make the next binaries. Wheeler later showed how to detect such a property by compiling the same source through an independent compiler lineage and comparing results [7].

Models

Machine-learning models increasingly help produce the data and supervision from which their successors are trained, through distillation, synthetic data, and automated evaluation. When models are trained recursively on the outputs of earlier models without sufficient fresh data, the lineage loses the tails of the original distribution and converges toward a narrower one, a phenomenon described as model collapse [8]. The lineage does not merely preserve properties; it erodes them.

5 What Separates Improvement from Drift

The three lineages differ in outcome: convergence to a precision no generation began with, invisible preservation of a property, and gradual loss. The difference does not lie in how much

each generation contributes to the next. It lies in the kind of check the lineage applies to its own outputs.

In the three-plate method, the check is relational and cannot be satisfied by agreement with oneself. A single plate compared only with its own copy can be convex and concave in matched pairs forever. Three plates compared in every pairing admit only one consistent shape. The criterion is external to any single generation, even though it is implemented entirely with the generation’s own products.

In the compiler lineage, the ordinary check is that the new compiler compiles its own source and produces a working binary. That check is satisfied by agreement with oneself, and Thompson’s construction exploits exactly that. Wheeler’s remedy works by adding a second, independent lineage, which reintroduces a comparison that one lineage cannot pass by self-agreement.

In recursive model training, the typical check is that successors perform well on evaluations that are themselves increasingly produced by the lineage. Where the reference distribution is replaced by the lineage’s own outputs, nothing outside the lineage constrains what is lost.

Proposition 1 (A condition for non-degenerate lineage). Let each generation be evaluated by a criterion V . If V can be satisfied by any generation whose outputs agree with its predecessor’s outputs, the lineage has no mechanism for correcting properties it inherits. Correction requires a criterion that at least some self-consistent but wrong generations fail.

Proof. If every self-agreeing generation passes V , then a generation that faithfully reproduces an inherited error passes V , and the error is not selected against. Conversely, if some self-agreeing but wrong generation fails V , the error has a way to be detected. $\square$

The proposition is almost trivial, which is its use. It turns a question about how much a lineage relies on itself into a question about what its checks can see. The three-plate method is powerful because its check is relational in a way self-agreement cannot fake. Thompson’s attack works because the compiler check is not. Model collapse occurs when the reference against which outputs are judged drifts along with the outputs.

6 Where the Ising Chip Sits

Optimization hardware for chip design sits on the favorable side of this line. An Ising machine’s output is a configuration, and its quality is judged by evaluating the Hamiltonian, or better, by checking the decoded route against the original constraints: are the pins connected, is the tree acyclic, are the segments rectilinear, what is the wire length. These checks are independent of the solver. The paper’s own classification of outcomes into optimal, suboptimal, and invalid trees is exactly such a check [1]. Beyond the solver, chip design has a long-established regime of sign-off verification, including design-rule checking and timing analysis, that judges a layout by criteria no router can satisfy by agreeing with itself. Many combinatorial problems of this kind share the structural advantage that checking a proposed answer is far easier than finding one.

A lineage of chips that route chips would therefore not be expected to drift in the manner of recursive model training. Its hazard is closer to Thompson’s. What a solver lineage can transmit invisibly is not a wrong answer, which the checks would catch, but a formulation: the choice of what counts as a violation, how heavily each constraint is weighted, which problems are mapped at all. The Shape of Search showed how much the penalty weights in this very routing formulation shape which configurations are reachable [11]. A formulation that is built

into one generation of tools, and then used to produce the benchmark instances and design flows on which the next generation is tuned, can persist without ever appearing wrong, because the checks it passes were written in its own terms. The remedy has the same shape as Wheeler’s: an independent formulation, or an independent tool lineage, against which the inherited one can be compared.

7 Conclusion

The scene of a chip solving chip-design problems, evaluated with an EDA company, is a real instance of type self-application and a possible beginning of lineage. It is not an instance of a technology designing itself, and at the demonstrated scale the problem it solves is already solved.

Lineage, however, is the relation that deserves attention, because it is the one through which tools shape their successors, and it is already pervasive in machine tools, compilers, and machine learning. Lineages differ not in how much each generation owes the last but in whether they contain a check that self-agreement cannot pass. Where such a check exists, as in the three-plate method and in the verification regime of chip design, a lineage can produce capabilities none of its generations began with. Where it does not, a lineage can carry forward, unseen, whatever its first generation assumed. No generation designs itself from nothing. What each generation can do is decide what the next one will be judged against, and that decision is where self-reference in technology has its consequences.

References

- [1] S. Li, Y. Zhang, A. Lee, Z. Zhu, L. Zeng, P. Wang, L. Gao, D. Wu, and W. Zhao. An Ising Machine for Combinatorial Optimization Based on Sub-Nanosecond CMOS-Integrated Magnetoresistive Random-Access Memory. *Nature Electronics*, 2026. doi:10.1038/s41928-026-01700-6.
- [2] M. R. Garey and D. S. Johnson. The Rectilinear Steiner Tree Problem Is NP-Complete. *SIAM Journal on Applied Mathematics*, 32:826–834, 1977.
- [3] M. Hanan. On Steiner’s Problem with Rectilinear Distance. *SIAM Journal on Applied Mathematics*, 14:255–265, 1966.
- [4] C. Chu and Y.-C. Wong. FLUTE: Fast Lookup Table Based Rectilinear Steiner Minimal Tree Algorithm for VLSI Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27:70–83, 2008.
- [5] W. R. Moore. *Foundations of Mechanical Accuracy*. Moore Special Tool Company, 1970.
- [6] K. Thompson. Reflections on Trusting Trust. *Communications of the ACM*, 27:761–763, 1984.
- [7] D. A. Wheeler. *Fully Countering Trusting Trust through Diverse Double-Compiling*. PhD thesis, George Mason University, 2009.
- [8] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal. AI Models Collapse When Trained on Recursively Generated Data. *Nature*, 631:755–759, 2024.

- [9] SpinX-Lab. Voltage-controlled-Spintronic-Ising-Machine: source data and plotting scripts. GitHub repository, 2026. <https://github.com/SpinX-Lab/Voltage-controlled-Spintronic-Ising-Machine>.
- [10] Flyxion. Replottable, Not Rerunnable: What a Figure Repository Lets a Reader Check. September 2026.
- [11] Flyxion. The Shape of Search: Reachability, Measurement, and Attraction in Physical, Cognitive, and Conversational Systems. September 2026.